

Lista Encadeada

Questão 2 · Lista simplesmente encadeada de inteiros

Enunciado

Implemente uma lista simplesmente encadeada de inteiros.

- Crie uma função para inserir no final.
- Crie uma função que receba um número informado pelo usuário e conte quantos valores iguais existem na lista. Retorne essa quantidade e informe na tela.
- Teste com os valores 5, 8, 5, 10, 8, 12, 5: insira-os, mostre a lista após a inserção e chame a função `contar_valores`.

Explicação passo a passo

Pense em um **trem**: cada vagão guarda um número e sabe qual é o próximo vagão. O último vagão aponta para NULL, que significa "fim da linha".

a) inserir_final

- 1 Cria um vagão novo com `malloc` e coloca o valor nele. O prox dele começa em NULL, porque será o último.
- 2 Se a lista está vazia, o vagão novo é o primeiro e a função o devolve como início.
- 3 Senão, anda até o último vagão (aquele cujo prox é NULL) e engata o vagão novo nele.
- 4 Devolve o início da lista, que só muda quando ela estava vazia.

b) contar_valores

Passeia vagão por vagão, do primeiro ao último, com um contador que começa em 0. Cada vez que o valor do vagão é igual ao procurado, soma 1. No fim, devolve o contador, e o main escreve o resultado na tela.

c) Teste

A lista fica 5 -> 8 -> 5 -> 10 -> 8 -> 12 -> 5 -> NULL. Ao buscar o 5, ele aparece 3 vezes (primeiro, terceiro e último vagão).

Código completo

Arquivo: 02_lista_encadeada.c. Para compilar: `gcc -Wall -o programa 02_lista_encadeada.c && ./programa`

```
/* Lista simplesmente encadeada de inteiros */
#include <stdio.h>
#include <stdlib.h>

typedef struct No {
    int valor;
    struct No* prox;
} No;

/* a) Inserir no final. Retorna o início (pode mudar se a lista estava vazia) */
```

```

No* inserir_final(No* inicio, int valor) {
    No* novo = (No*) malloc(sizeof(No));
    if (!novo) { printf("Sem memoria!\n"); return inicio; }
    novo->valor = valor;
    novo->prox = NULL;

    if (inicio == NULL) return novo;          /* lista vazia: o novo é o primeiro */

    No* p = inicio;
    while (p->prox != NULL) p = p->prox;      /* anda até o último vagão */
    p->prox = novo;                            /* engata o novo vagão no final */
    return inicio;
}

void mostrar_lista(No* inicio) {
    printf("Lista: ");
    for (No* p = inicio; p; p = p->prox)
        printf("%d%s", p->valor, p->prox ? " -> " : " -> NULL");
    printf("\n");
}

/* b) Conta quantas vezes 'alvo' aparece */
int contar_valores(No* inicio, int alvo) {
    int cont = 0;
    for (No* p = inicio; p; p = p->prox)
        if (p->valor == alvo) cont++;
    return cont;
}

void liberar_lista(No* inicio) {
    while (inicio) { No* p = inicio->prox; free(inicio); inicio = p; }
}

int main(void) {
    No* lista = NULL;
    int valores[] = {5, 8, 5, 10, 8, 12, 5};    /* c) valores do teste */
    int n = sizeof(valores) / sizeof(valores[0]);

    for (int i = 0; i < n; i++) lista = inserir_final(lista, valores[i]);

    mostrar_lista(lista);

    int busca;
    printf("Digite um numero para contar: ");
    scanf("%d", &busca);
    printf("O numero %d aparece %d vez(es) na lista.\n", busca, contar_valores(lista, busca));

    liberar_lista(lista);
    return 0;
}

```

Teste executado

Valor digitado pelo usuário: 5.

Lista: 5 -> 8 -> 5 -> 10 -> 8 -> 12 -> 5 -> NULL

Digite um numero para contar: 0 numero 5 aparece 3 vez(es) na lista.